

---

## Exercice 1 — Classe & Encapsulation (4 pts)

On souhaite modéliser une classe `Etudiant`.

### Travail demandé :

1. Déclarer une classe avec les attributs privés :
  - `nom` (string)
  - `note` (float)
2. Écrire :
  - un constructeur
  - les getters / setters
3. Ajouter une méthode `afficher()` qui affiche les informations

---

## Exercice 2 — Allocation dynamique (2 pts)

1. Créer un objet `Etudiant` dynamiquement
2. Afficher ses informations
3. Libérer la mémoire

---

## Exercice 3 — Surcharge d'opérateur (4 pts)

Créer une classe `Complexe` représentant un nombre complexe.

### Travail demandé :

1. Attributs : `re`, `im`
2. Constructeur
3. Surcharger :

- l'opérateur `+`
  - l'opérateur `<<` pour affichage
- 

## Exercice 4 — Héritage (4 pts)

On considère :

- une classe `Vehicule`
- une classe `Voiture` qui hérite de `Vehicule`

**Travail demandé :**

1. `Vehicule` contient :
    - attribut `marque`
    - méthode `afficher()`
  2. `Voiture` contient :
    - attribut `nbPortes`
  3. Redéfinir la méthode `afficher()`
- 

## Exercice 5 — Polymorphisme (3 pts)

Modifier les classes précédentes pour :

1. Rendre la méthode `afficher()` virtuelle
  2. Créer un tableau de pointeurs de `Vehicule`
  3. Y stocker des objets `Vehicule` et `Voiture`
  4. Afficher tous les éléments (polymorphisme)
- 

## Exercice 6 — Classe abstraite (3 pts)

Créer une classe abstraite `Forme`.

**Travail demandé :**

1. Déclarer une méthode virtuelle pure `aire()`

2. Créer :
    - une classe `Cercle`
    - une classe `Rectangle`
  3. Implémenter `aire()` dans chaque classe
- 

## Exercice 7 — Problème complet (Bonus 2 pts)

On souhaite gérer un système bancaire.

### Travail demandé :

1. Créer une classe `Compte` :
  - attribut `solde`
  - méthodes `deposer()` et `retirer()`
2. Créer une classe `CompteEpargne` :
  - héritage de `Compte`
  - attribut `taux`
3. Ajouter une méthode pour appliquer les intérêts